Banking Operations Incident Detection

Capstone v1: predicting critical incident hours from system KPIs
FullStack Academy AI/ML | Cohort 2605-FTB-CT-AIM-PT

The problem

- Banks run NEFT, IMPS, and RTGS transfers around the clock.
- When a system overloads, customers feel it before the team sees it.
- Monitoring tools track business metrics, logs, and infrastructure separately, so root cause takes longer to find.
- Goal for v1: flag critical incident hours from the same KPIs the operations team already watches.

The data

- incident_reports.csv: 168 hourly rows, 2025-10-20 to 2025-10-26.
- Labels: 142 normal (Normal and Resource_Warning), 26 critical (Resource_Exhaustion).
- Features: CPU, memory, DB connection pool, application error rate, NEFT success rate.
- infrastructure_metrics.csv: 10,080 minute-level rows, same week. Checked for missing values, duplicates, and gaps: none found.

Critical hours look very different

KPI (hourly mean)	Normal (142 h)	Critical (26 h)
CPU utilization (%)	48.9	**89.8**
Memory usage (%)	43.9	**85.4**
DB connection pool (%)	39.3	**89.6**
Application error rate	2.05	**14.54**
NEFT success rate (%)	98.0	**87.3**

Every KPI separates the two groups by a wide margin.

Finding 1: the labels follow a simple rule

- Rule: flag an hour as critical if NEFT success < 95%.
- On the 168 incident hours, this rule matches the is_critical label exactly.
- Lowest normal NEFT value: 96.0%. Highest critical NEFT value: 89.8%.
- Implication: a model that uses NEFT will score near-perfectly, and that score says little about learning. The rule is the honest baseline.
- Open question for instructor: were labels set by rule or from real incident records?

Finding 2: the two data files disagree

- 142 of 168 incident rows match infrastructure_metrics.csv exactly.
- All 26 critical rows do not match. At those hours:
- Incident file: NEFT about 87%, CPU about 90%.
- Infrastructure file: NEFT about 98%, CPU about 35 to 45% in most of those hours.
- v1 uses only incident_reports.csv. The conflict goes in Data Limitations and needs a source answer before results are final.

Approach: baseline first, then one model

- Time-based split: first 5 days for training (120 h, 17 critical); last 2 days for testing (48 h, 9 critical).
- Baseline: threshold rule on NEFT success (critical if < 95%). No learning.
- Model: logistic regression with standardized features, balanced class weights.
- Run twice: with NEFT included, and without NEFT. The without-NEFT run tests whether other KPIs carry the signal.
- Metrics: precision, recall, F1, and a confusion matrix, because the test set has only 9 critical hours.

Results: same-hour detection vs. early warning

Test (48 hours, 9 critical)	Precision	Recall	F1
Same hour: NEFT < 95% rule	1.000	1.000	1.000
Same hour: logistic, all KPIs	1.000	1.000	1.000
Same hour: logistic, without NEFT	1.000	1.000	1.000
Early warning: previous hour NEFT < 95%	0.000	0.000	0.000
Early warning: logistic, lagged KPIs	**0.214**	**0.333**	**0.261**

Same-hour scores are perfect because one KPI threshold separates the labels. The early-warning result is the real test: 3 of 9 critical hours caught, 11 false alarms.

Limitations

- Small data: 168 hours, 26 critical. Each missed critical hour changes recall noticeably.
- Labels may be rule-derived, so high scores may reflect the labeling rule, not system behavior.
- Two sources disagree at the critical hours.
- Hourly resolution only. The minute-level file was not used for labeling.
- Single week of data. No evidence about seasonality or longer-term drift.
- Early warning is weak: the lagged model misses 6 of 9 critical hours.

Next steps

- Confirm label source and authoritative file with the instructor.
- Test whether the labels hold up with the instructor and the authoritative data source.
- If time allows: minute-level anomaly flags, a simple Streamlit alert view.
- Stretch (not in v1): log clustering with TF-IDF, LLM root cause summaries, FastAPI, Docker.